

Tata Group: AI Legal Document Intelligence System

1. Executive Overview

Tata Group operates across a large and diverse enterprise footprint where legal and compliance teams are expected to review contracts, vendor agreements, policy documents, regulatory filings, and internal compliance records with consistency, speed, and defensible judgment. Review volume is increasing across business units, while the need for clause-level precision, obligation visibility, and risk traceability is becoming more critical for leadership decision-making.

The AI Legal Document Intelligence System is designed to reduce the manual burden of first-pass legal document review while preserving human authority over final interpretation and approval. The system converts uploaded documents into structured legal intelligence by applying OCR, document parsing, clause extraction, retrieval-augmented generation, legal reasoning, summarization, and risk flagging in a governed workflow.

The product solves a practical business problem: legal teams need a faster way to identify key clauses, compare obligations, surface unusual terms, summarize agreement positions, and route high-risk documents for senior review. The expected output is Contract summaries & risk flags that help legal and compliance teams focus attention where judgment is most needed.

AI and system design support the Legal / Compliance function by creating a traceable document-intelligence pipeline rather than a generic text-generation tool. Each document is processed through defined stages: upload, OCR, parsing, clause extraction, RAG retrieval, legal reasoning, and summarization. The architecture makes it possible to retain source references, confidence signals, approval status, reviewer feedback, and audit logs for operational control.

The release matters now because enterprise legal teams are under pressure to improve turnaround time without weakening governance. Faster summaries are valuable only if they are grounded in document text, validated against approved references, and reviewable by legal professionals. The first release must therefore demonstrate that AI can accelerate legal operations while keeping approval, accountability, and escalation mechanisms intact.

2. Why This Release Is Time-Critical

The immediate trigger is the scale and distribution of legal work across Tata Group business units. Contract review, vendor onboarding, policy interpretation, and compliance documentation often move through different teams, systems, and review patterns. When document volume increases, inconsistent first-pass review can delay business decisions and create avoidable risk exposure.

Tata Group cannot delay the release because slow document review directly affects procurement, partnerships, vendor negotiations, regulatory preparedness, and internal control. A delayed or inconsistent review process can allow unfavorable obligations, vague indemnity language, missing termination rights, unusual liability caps, privacy exposure, or compliance gaps to reach later approval stages before they are detected.

The strategic pressure is not only operational speed. The organization needs a common intelligence layer that can help legal teams compare documents against approved positions, detect recurring clause deviations, and escalate material risks before they become negotiation or compliance problems. A first release that proves clause extraction, risk flagging, and legal-summary generation will create a repeatable foundation for broader legal-technology modernization.

Leadership expects the first release to be narrow enough to ship reliably and strong enough to be trusted. It must accept uploaded documents, extract readable text, identify important clauses, retrieve relevant reference context, generate review-ready summaries, flag high-risk terms, and preserve human approval. The timeline is short because legal throughput and governance consistency are already active business concerns, not future optimization opportunities.

3. Leadership Expectations

Outcome Area	Expectation	Why It Matters
Review Velocity	Reduce the time required to produce first-pass contract summaries and risk flags from uploaded legal documents.	Legal and compliance teams can focus scarce expert time on judgment, negotiation strategy, and escalations.
Clause-Level Accuracy	Extract key clauses, obligations, dates, parties, liability terms, termination provisions, confidentiality language, and compliance references with traceability.	The product must help teams review actual document obligations rather than relying on broad summaries.
Grounded Legal Reasoning	Use RAG retrieval against approved clause libraries, policies, prior agreement positions, and compliance references before generating conclusions.	Leadership needs confidence that AI outputs are anchored in document text and approved context.
Risk Flag Quality	Identify unusual, missing, conflicting, or high-risk clauses and explain why each item requires attention.	Risk flags must guide review prioritization instead of creating noise for legal teams.
Human Approval Control	Keep legal professionals in control of acceptance, edits, escalation, and final approval status.	The system accelerates analysis but cannot replace accountable legal review.
Enterprise Auditability	Preserve document history, extracted clauses, model outputs, reviewer actions, and approval decisions.	Audit trails are essential for governance, compliance, and cross-business-unit accountability.
Deployment Readiness	Deliver a working internal product with clear workflow visibility, documentation, and demonstration readiness.	Leadership needs a reviewable product that can be assessed as a serious enterprise capability.

4. Leadership Meeting Transcript

The following transcript captures a leadership, legal, compliance, product, engineering, and AI delivery discussion for the AI Legal Document Intelligence System. Only designations are used to maintain a professional internal-document tone.

Chief Legal Officer: The review burden is becoming material across business units. We have contracts, vendor agreements, filings, policies, and compliance records moving through different teams, and the first-pass review effort is too manual. We need faster visibility into obligations and risks, but we cannot compromise legal judgment or accountability.

Chief Technology Officer: The product must be designed as a governed document-intelligence system, not a generic document chatbot. If a user uploads an agreement, the system should process the document, extract clauses, retrieve approved context, reason over the obligation, and show the evidence trail. We need structured workflow stages and auditability.

Chief Compliance Officer: My concern is consistency. A clause may look acceptable in one business unit and risky in another depending on regulatory exposure, data handling, vendor criticality, and geography. The system should flag risk with context and make it easy for reviewers to see why an item was highlighted.

Business Head: The operational impact is real. Vendor onboarding and commercial negotiations can slow down because teams wait for legal review. Faster first-pass summaries would help, especially if the system can show key obligations, missing clauses, and escalation points early in the process.

Chief Product Officer: The product should have a clear first release scope. We should focus on uploaded documents, OCR, parsing, clause extraction, retrieval from approved references, legal reasoning, summaries, and risk flags. We should not attempt to automate final legal decisions.

Product Manager: The primary user journey starts with a legal team member uploading a document. The system processes the file, shows document-readiness status, extracts key clauses, compares them against known positions, generates a summary, and routes uncertain or high-risk items for human review.

Legal Operations Lead: We need workflow controls. Reviewers should be able to accept a summary, edit a clause interpretation, add comments, escalate to a senior reviewer, or mark the output as not usable. The product should record every action.

Engineering Lead: From an architecture perspective, each uploaded document should become a processing job. The job moves through upload, OCR, parsing, clause extraction, retrieval, reasoning, summarization, risk classification, and review. Each stage should store structured outputs and errors.

AI Engineer: The AI layer should be constrained by document evidence and retrieved legal context. We can use extraction prompts for clauses, retrieval prompts for relevant precedent or policy context, and reasoning prompts that produce structured JSON with risk category, rationale, confidence, and citations.

Data Scientist: Evaluation is critical. We need test documents where expected clauses and known risks are defined. Metrics should include clause recall, risk-flag precision, summary faithfulness, citation accuracy, and reviewer acceptance rate.

Data Analyst: The data model should capture document metadata, page OCR confidence, clause records, obligation records, retrieval references, generated summaries, risk flags, reviewer decisions, and processing logs. This will allow legal operations to track review throughput and recurring risk patterns.

Backend Engineer: The API design should separate document storage from processing logic. We need endpoints for upload, OCR job creation, clause extraction, retrieval, summary generation, review actions, and audit retrieval. Secrets and document files must stay backend-controlled.

Frontend Engineer: The interface should make the workflow transparent. Users need an upload workspace, a processing-status view, a clause review panel, a risk summary dashboard, and a final report workspace. The product should not hide the source document behind a black-box answer.

Security Lead: Legal documents can contain commercially sensitive, personal, and confidential information. We need access controls, encrypted storage, restricted logs, secure environment variables, and clear policies around what text is sent to model services.

Compliance Lead: The output must clearly indicate that it is system-generated and pending legal review until approved. We need a visible approval state, evidence references, and a way to export audit-ready review records.

QA Lead: Testing should include scanned documents, poor OCR quality, long contracts, missing clauses, duplicate clauses, conflicting obligations, unusual governing-law provisions, and documents with tables or annexures. Failures should be visible and actionable.

Operations Lead: For the first release, we need a reliable demonstration flow: upload a vendor agreement, process it, extract clauses, flag risks, generate a summary, and complete human approval. The product must feel stable enough for leadership review.

Chief Technology Officer: Agreed. The first release should prove the full workflow. We can expand later into negotiation playbooks, obligation monitoring, policy mapping, and integration with enterprise document repositories, but the foundation must be traceable document intelligence with human approval.

5. Product Blueprint

5.1 Product Vision

document repositories, but the foundation must be traceable document intelligence with human approval.

5. Product Blueprint

5.1 Product Vision

The product vision is to give Tata Group legal and compliance teams a governed AI document-intelligence environment that can move from uploaded legal documents to Contract summaries & risk flags with speed, evidence traceability, and human approval. The AI Legal Document Intelligence System should unify OCR, NLP pipelines, vector retrieval, and LLM reasoning into a controlled review workflow that supports legal professionals instead of bypassing them.

5.2 Product Surfaces

- Document Upload Workspace for submitting contracts, vendor agreements, policy documents, regulatory filings, and internal compliance records.
- OCR and Parsing Monitor for tracking page-level extraction quality, failed pages, document sections, and processing status.
- Clause Intelligence Panel for viewing extracted clauses, obligations, counterparties, dates, renewal terms, termination rights, liability terms, and compliance references.
- Risk Review Console for surfacing missing clauses, unusual terms, conflicting obligations, high-risk language, confidence levels, and reviewer decisions.
- RAG Knowledge Workspace for retrieving approved clause libraries, prior positions, policy references, regulatory context, and internal guidance.
- Legal Summary Workspace for generating executive summaries, obligation maps, risk narratives, comparison notes, and export-ready review outputs.
- Legal Operations Dashboard for monitoring document volume, review turnaround, risk categories, escalation rate, and approval outcomes.

Product Surface Map
AI Legal Document Intelligence System

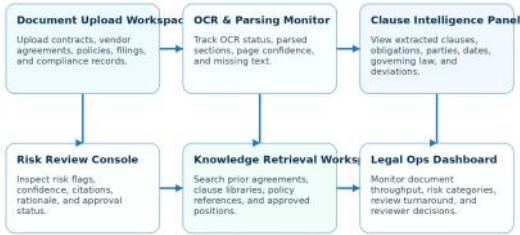


Figure: Product surface map for legal review, compliance governance, and legal operations workflows.

5.3 Primary User Journeys

Journey	Typical User Need	System Behavior	Expected Outcome
Contract First-Pass Review	A legal reviewer needs a quick understanding of a new vendor or commercial agreement.	System runs OCR, parsing, clause extraction, retrieval, reasoning, and summary generation.	Review-ready contract summary with key terms, obligations, and risk flags.
Clause Risk Detection	A compliance team needs to identify unusual or missing clauses before approval.	System compares extracted clauses against approved references and known risk patterns.	Prioritized risk list with rationale, confidence, and cited document locations.
Obligation Comparison	A business unit needs to compare obligations across vendor agreements.	System extracts obligations, parties, dates, service commitments, renewal terms, and termination conditions.	Structured obligation comparison that supports negotiation and compliance tracking.
Policy and Filing Review	A compliance team needs to summarize internal policies or regulatory documents.	System parses long documents, retrieves relevant policy context, and generates section-level summaries.	Concise summary with compliance themes, affected teams, and follow-up actions.
Senior Legal Escalation	A reviewer needs to escalate high-risk or uncertain items to senior counsel.	System packages clause evidence, risk rationale, source references, and reviewer notes.	Escalation-ready review record with clear decision context.

Figure: Legal document review journey from upload to human approval.

5.4 Detailed Product Requirements

Document Intake and Upload Handling

- The system must accept PDF, DOCX, scanned document images, and text-based legal files where technically feasible.
- The upload workflow must capture document type, business unit, counterparty, geography, confidentiality level, and review priority.
- The product should validate file size, supported format, upload completion, and document-readiness status before processing.
- Users should see clear failure messages for unreadable files, unsupported formats, or incomplete uploads.

OCR and Parsing Pipeline

- OCR must convert scanned pages into text and store page-level confidence, page numbers, and extraction status.
- The parsing layer should identify headings, sections, tables, annexures, signature blocks, and key metadata.
- Low-confidence pages must be flagged for manual review rather than treated as reliable input.
- Parsed text should preserve document location references for clause-level citation and audit review.

Clause Extraction and Normalization

- The system must extract key clauses such as indemnity, limitation of liability, confidentiality, termination, renewal, payment, data protection, governing law, dispute resolution, audit rights, and compliance obligations.
- Each clause record should include clause type, extracted text, page or section reference, involved party, obligation owner, and confidence score.
- The product should detect missing expected clauses based on document type and business context.
- Clause outputs must be stored in structured formats that support review, comparison, and downstream analytics.

RAG Retrieval and Knowledge Context

- The retrieval layer must search approved clause libraries, internal policy references, prior agreement positions, and compliance guidance.
- Retrieved context should be ranked by relevance, document type, business unit, jurisdiction, recency, and approval status where metadata is available.
- The system should expose retrieved references used by the reasoning layer so reviewers can validate the basis of each recommendation.
- Outdated or unapproved references should be marked clearly and excluded from high-confidence reasoning.

Legal Reasoning and Risk Flagging

- The LLM reasoning layer must generate risk flags only when supported by extracted document text or retrieved approved context.
- Risk flags should include severity, rationale, affected clause, source location, recommended reviewer action, and confidence.
- The system should distinguish missing information, ambiguous language, non-standard wording, conflicting obligations, and high-risk provisions.
- Outputs must avoid presenting legal conclusions as final decisions until approved by authorized legal reviewers.

Summarization and Review Output

- The summary generator must produce concise contract summaries, obligation maps, risk sections, and approval notes.
- Summaries should include cited clause references and indicate where OCR or extraction confidence is low.
- Users must be able to export review outputs into a shareable format suitable for internal decision-making.
- Generated summaries should be editable before approval and preserve version history.

Human Review and Approval Layer

- Reviewers must be able to accept, edit, reject, escalate, or comment on AI-generated summaries and risk flags.
- Approval status should be visible at document, clause, and risk-flag levels where appropriate.
- The system should capture reviewer identity, timestamp, decision, and rationale for audit purposes.
- High-risk or low-confidence outputs should be routed to designated reviewers before final acceptance.

Security, Logging, and Observability

- Access control must be aligned to document sensitivity, business unit, and reviewer role.
- Operational logs should capture upload events, processing stages, model calls, retrieval references, failures, retries, and reviewer actions.

- High-risk or low-confidence outputs should be routed to designated reviewers before final acceptance.

Security, Logging, and Observability

- Access control must be aligned to document sensitivity, business unit, and reviewer role.
- Operational logs should capture upload events, processing stages, model calls, retrieval references, failures, retries, and reviewer actions.
- Sensitive document content should not be exposed in frontend logs or uncontrolled telemetry.
- Monitoring dashboards should track processing time, failure rates, OCR quality, clause extraction coverage, and reviewer acceptance rates.

5.5 Non-Functional Expectations

Area	Expectation	Notes
Reliability	The system must avoid unsupported legal interpretations and expose confidence, source location, and missing-context indicators.	Human approval and evidence traceability are essential for legal and compliance use.
Performance	Document processing should complete within a practical review window for typical agreements and policy files.	Use asynchronous processing, background jobs, chunking, caching, and progressive status updates.
Security	Legal documents, commercial terms, personal data, API keys, and model credentials must be protected.	Use encrypted storage, backend-only secrets, access controls, audit logs, and secure environment variables.
Maintainability	OCR, parsing, extraction, retrieval, reasoning, summarization, review, and audit services should be modular.	Each workflow stage must be independently testable and replaceable.
Usability	Legal reviewers should understand what the system extracted, why it flagged risk, and what requires approval.	Use clear clause views, citations, confidence indicators, and reviewer actions.
Scalability	The architecture should support multiple business units, concurrent document jobs, and reusable legal knowledge assets.	Use job queues, structured stores, vector search, and metadata-aware retrieval.
Observability	Every document job, extraction result, retrieval reference, model output, reviewer action, and failure should be traceable.	Legal operations and engineering teams need debugging, quality, and audit visibility.
Deployment Readiness	The product must be deployable with stable public or controlled internal access, environment configuration, and clear documentation.	Leadership evaluation requires repeatable setup, a working demo flow, and accessible review materials.
Cost Optimization	The platform should control OCR, model, retrieval, storage, compute, and monitoring costs without weakening reliability or governance.	Use batch processing, caching, tiered model routing, token-budget controls, retention policies, usage dashboards, and cost alerts for expensive document workflows.

6. Technical Architecture

The architecture is organized around a legal document job lifecycle. A document is uploaded, converted into machine-readable text through OCR, parsed into sections, processed for clause extraction, enriched through RAG retrieval, analyzed through constrained legal reasoning, summarized into review-ready output, and routed through human approval with audit capture.

Primary Legal Review Journey

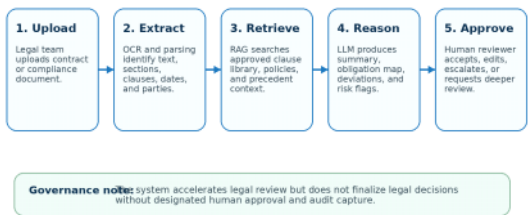


Figure: Technical architecture for the AI Legal Document Intelligence System.

6.1 Architecture Narrative

The workflow begins when a legal team member uploads a contract, agreement, policy document, filing, or compliance record. The intake layer records document metadata, file type, business unit, review priority, confidentiality category, and target workflow. The uploaded file is stored securely and a document-processing job is created. The OCR and parsing layer converts scanned or image-based pages into text, identifies sections, detects tables and annexures, and stores page-level quality signals. The clause extraction layer then identifies key provisions and normalizes them into structured records with clause type, extracted text, page reference, involved parties, obligations, and confidence values. The retrieval layer indexes approved clause libraries, internal policies, prior positions, compliance references, and document embeddings in a vector database. When a clause requires interpretation or comparison, the RAG workflow retrieves relevant references and passes them to the reasoning layer with the extracted document evidence. The LLM reasoning layer generates summaries, obligation maps, deviation notes, and risk flags using structured prompts and constrained output schemas. It must cite extracted document locations and retrieved context for material conclusions. The approval layer allows legal reviewers to accept, edit, reject, escalate, or annotate outputs while preserving the audit trail. The backend manages document jobs, API calls, model calls, storage, retrieval, processing state, error handling, and logs. The frontend provides upload, review, risk, summary, and operations views. Deployment should include secure environment variables, controlled access, working application links, and clear documentation for ongoing review and maintenance.

6.2 Recommended Technology Choices

Layer	Preferred Options	Why It Fits
Frontend/Product Layer	Next.js, React, Tailwind CSS	Supports a polished document upload workspace, clause review panels, dashboards, and approval workflows.
Backend/API Layer	FastAPI, Node.js/Express, Next.js API routes	Manages document jobs, upload APIs, extraction endpoints, retrieval calls, reasoning requests, reviewer actions, and audit logging.
OCR Layer	Tesseract OCR, PaddleOCR, EasyOCR	Provides text extraction from scanned contracts, policy documents, and image-based legal files.
NLP and Parsing Layer	spaCy, PyMuPDF, pdfplumber, Unstructured	Useful for parsing documents, extracting sections, processing PDFs, and preparing text chunks.
Vector Database	Chroma, Qdrant, Supabase pgvector	Stores embeddings for clause libraries, prior agreement positions, policy references, and document chunks.
LLM Layer	Gemini API, local/open-source LLM options where appropriate	Supports clause reasoning, risk explanation, structured summarization, and review-note generation.
RAG and Orchestration	LangChain, LangGraph, LlamaIndex, CrewAI	Coordinates retrieval, tool calls, clause extraction, reasoning, and structured outputs.
Operational Data Store	PostgreSQL / Supabase	Stores document jobs, extracted clauses, risk flags, reviewer decisions, metadata, and audit records.
Deployment	Vercel, Netlify, Render, Railway	Enables frontend and backend deployment with environment variables and accessible demonstration links.
Monitoring and Logging	OpenTelemetry, application logs, structured audit tables	Supports traceability, debugging, governance review, and operational reporting.

6.3 Data Assets Required

- Uploaded document records containing file name, file type, business unit, document category, confidentiality level, counterparty, jurisdiction, and review priority.
- OCR output containing extracted text, page numbers, OCR confidence, unreadable regions, table indicators, and processing status.
- Parsed document sections containing headings, clause boundaries, annexures, signature blocks, defined terms, and relevant metadata.
- Clause records containing clause type, extracted language, obligation owner, affected party, date, renewal or termination condition, reference location, and confidence score.
- Knowledge base assets containing approved clause library, internal policy references, compliance rules, prior approved positions, risk taxonomy, and review playbooks.
- Vector index records containing embeddings for document chunks, clause references, approved policies, and historical summaries.
- Risk and summary records containing AI-generated risk flags, rationale, severity, cited evidence, reviewer decision, escalation status, and version history.

- Operational logs containing upload events, processing stages, model requests, retrieval references, failures, retries, latency, and reviewer actions.

6.4 Deployment Flow

The product should be maintained in a GitHub repository with separate folders for frontend, backend, document-processing services, AI workflows, data schemas, documentation, tests, and deployment notes. The frontend can be deployed through Vercel or Netlify. The backend can be deployed through a hosted API service such as Render, Railway, or serverless API routes when appropriate. The database and vector database should be provisioned separately so document metadata, clause records, summaries, risk flags, and embeddings can be managed with clear access controls. Environment variables must be used for model keys, storage credentials, database URLs, vector database keys, OCR settings, and authentication secrets. The public or controlled app URL should be documented in the README along with setup steps, screenshots, architecture notes, API details, and demonstration flow.

7. Detailed Solution Plan and Build Approach

7.1 Repository and Folder Structure

tata-ai-legal-document-intelligence-system/

```
├─ README.md
├─ .env.example
├─ frontend/
│   └─ app/
│       └─ components/
│       └─ pages/
│       └─ styles/
├─ backend/
│   └─ api/
│   └─ auth/
│   └─ services/
│   └─ storage/
│   └─ database/
│       └─ audit/
├─ document_pipeline/
│   └─ ocr/
│   └─ parsing/
│   └─ clause_extraction/
│   └─ normalization/
│       └─ quality_checks/
├─ ai_workflows/
│   └─ rag_retrieval/
│   └─ legal_reasoning/
│   └─ risk_flagging/
│   └─ summarization/
│   └─ prompt_templates/
│       └─ output_schemas/
├─ data/
│   └─ sample_contracts/
│   └─ clause_library/
│   └─ policy_references/
│   └─ risk_taxonomy.csv
│   └─ sample_review_records.csv
├─ docs/
│   └─ architecture.md
│   └─ api_documentation.md
│   └─ security_notes.md
│   └─ demo_script.md
│       └─ screenshots/
├─ tests/
│   └─ functional_tests/
│   └─ document_processing_tests/
│   └─ ai_output_tests/
│       └─ edge_cases/
├─ deployment/
├─ vercel_notes.md
├─ backend_deployment.md
└─ environment_setup.md
```

7.2 Data and Knowledge Preparation Approach

Start by preparing representative legal document categories such as vendor agreements, service agreements, NDAs, internal policies, regulatory filings, and compliance records. Each sample should include metadata such as document type, business unit, jurisdiction, counterparty, review priority, and confidentiality level.

Clean the data by separating raw documents from extracted text, parsed sections, clause records, and review outputs. OCR results should be stored with page-level confidence so low-quality extraction can be routed for human review. Clause records should use a consistent schema for clause type, document location, extracted text, obligation owner, risk status, and reviewer decision.

Prepare a controlled knowledge base containing approved clause templates, internal legal positions, compliance policy references, risk taxonomy, and review guidance. This content should be chunked, embedded, indexed in a vector database, and tagged with metadata such as document type, business unit, jurisdiction, version, approval date, and owner. The AI workflow should use this knowledge base only as context, not as a substitute for legal approval.

7.3 Backend and API Approach

- POST /api/documents/upload: upload a legal document, store metadata, and create a processing job.
- GET /api/documents/{document_id}/status: retrieve OCR, parsing, extraction, reasoning, and review status.
- POST /api/ocr/run: execute OCR for scanned or image-based documents and store page-level confidence.
- POST /api/parse: convert OCR or text-based documents into sections, tables, annexures, and metadata.
- POST /api/clauses/extract: extract key clauses, obligations, parties, dates, and confidence scores.
- POST /api/rag/retrieve: retrieve approved clause references, policies, and prior positions relevant to extracted clauses.
- POST /api/reasoning/risk-flags: generate structured risk flags with rationale, severity, evidence, and confidence.
- POST /api/summaries/generate: generate contract summaries, obligation maps, and review notes.
- POST /api/review/actions: capture reviewer edits, approvals, rejections, escalations, and comments.
- GET /api/audit/{document_id}: retrieve document processing history, reviewer actions, and evidence trail.

Authentication should be required for upload, review, and audit actions. Backend services should validate file type, document size, user permissions, and confidentiality level before processing. Error handling should return clear messages for OCR failure, missing text, retrieval failure, model timeout, unsupported format, or low-confidence outputs. Logs should be structured and should avoid exposing sensitive legal text unless explicitly stored in approved audit tables.

7.4 AI Workflow Approach

Upload Documents: Capture document files and metadata, create a processing job, and store the original file securely.

OCR: Convert scanned pages into text, record page-level confidence, and flag unreadable or incomplete pages.

Parsing: Identify document sections, clause boundaries, definitions, tables, annexures, signature blocks, and important metadata.

Clause Extraction: Extract key legal clauses and convert them into structured records with clause type, source location, involved parties, and confidence.

RAG Retrieval: Search approved clause libraries, internal policy references, prior legal positions, and compliance guidance relevant to the extracted clause.

Legal Reasoning: Compare extracted clauses against retrieved context and generate structured risk flags, deviation notes, missing-clause indicators, and suggested reviewer actions.

Summarization: Produce review-ready summaries, obligation maps, risk sections, and exportable approval notes with evidence references.

7.5 Prompting / Reasoning / Agentic Approach

Prompts should be structured around evidence, constraints, and output schema. Clause extraction prompts should require the model to return clause type, exact extracted text, source page, obligation owner, affected party, date, and confidence. Retrieval prompts should define the relevant knowledge categories to search, such as approved clause language, internal policy guidance, prior legal positions, and compliance risk taxonomy.

The reasoning workflow should use specialized roles where useful: a clause extraction agent, retrieval agent, risk classification agent, obligation mapping agent, summary writer, and review-preparation agent. Tool calling should be used for OCR status, vector search, document retrieval, clause lookup, and audit storage. The memory layer should store approved references, not uncontrolled user-generated legal conclusions.

Validation should check that every material risk flag references extracted document text and, where applicable, retrieved approved context. Structured outputs should use JSON schemas for clause records, risk flags, summaries, obligation maps, and reviewer actions. Human review is required before summaries or risk determinations are treated as approved work products.

7.6 Frontend Product Approach

- Document Upload screen with file selection, document metadata, confidentiality indicator, and review-priority field.
- Processing Status screen showing OCR, parsing, clause extraction, retrieval, reasoning, summarization, and review stages.
- Clause Intelligence screen displaying extracted clauses, page references, confidence scores, and comparison context.
- Risk Review screen showing risk severity, rationale, affected clause, retrieved references, and reviewer action buttons.
- Summary Workspace showing contract overview, key obligations, dates, parties, missing clauses, and recommended follow-up actions.
- Approval and Escalation screen enabling accept, edit, reject, escalate, comment, and export actions.
- Legal Operations Dashboard showing throughput, high-risk categories, processing failures, average review time, and acceptance rate.

7.7 Testing Approach

- Functional tests should verify upload, OCR job creation, parsing, clause extraction, retrieval, reasoning, summarization, review actions, and audit retrieval.
- Document processing tests should include text-based PDFs, scanned PDFs, DOCX files, long agreements, annexures, tables, poor-quality scans, and mixed-format documents.
- AI output tests should evaluate clause recall, summary faithfulness, risk-flag precision, citation accuracy, structured-output validity, and refusal behavior when evidence is insufficient.
- Edge cases should include missing clauses, conflicting clauses, unreadable pages, duplicate sections, unusual governing-law provisions, unsupported document types, retrieval failure, and model timeout.
- Failure handling should show meaningful user messages and preserve partial processing outputs where useful.
- Demo scenarios should include a vendor agreement review, a policy-document summary, and a high-risk clause escalation flow.

7.8 Deployment Approach

Deploy the frontend through Vercel or Netlify and deploy the backend through a hosted API service such as Render, Railway, or serverless API routes where the processing workload fits. Store all model, OCR, database, vector database, storage, and authentication credentials as environment variables. Provide a .env.example file with placeholder names, not real secrets.

The README must include the public or controlled application URL, setup steps, architecture explanation, API documentation, screenshots, sample document workflow, security notes, and demonstration script. The demo video should show the full journey from document upload to OCR, clause extraction, RAG retrieval, risk flag generation, summary review, and human approval.

8. Reference Stack and Useful Links

Category	Tool / Resource	Link	Why It Is Useful
Provided GitHub Reference	Paralegal	https://github.com/clusterknot/ai-engineering-hub/tree/main/paralegal-agent-crew	Supports role-based legal-assistant and document-review agent patterns.
Provided GitHub Reference	Pipeline	https://github.com/clusterknot/ai-engineering-hub/tree/main/groundX-doc-pipeline	Useful reference for document ingestion, processing, and pipeline-style knowledge preparation.
Provided GitHub Reference	Context	https://github.com/clusterknot/ai-engineering-hub/tree/main/context-engineering-workflow	Useful for designing context preparation, retrieval grounding, and structured reasoning workflows.
Official Documentation	Tesseract OCR	https://tesseract-ocr.github.io/	OCR option for extracting text from scanned legal documents.
Official Documentation	PaddleOCR	https://github.com/PaddlePaddle/PaddleOCR	OCR toolkit for document images and scanned pages.
Official Documentation	EasyOCR	https://github.com/JaidedAI/EasyOCR	OCR option for multilingual and image-based document extraction.
Official Documentation	spaCy	https://spacy.io/usage	NLP pipeline support for entity recognition, text processing, and clause-preparation workflows.
Official Documentation	PyMuPDF	https://pymupdf.readthedocs.io/	PDF parsing and extraction support for legal document workflows.
Official Documentation	pdfplumber	https://github.com/jsvine/pdfplumber	Useful for extracting text and tables from PDF documents.
Official Documentation	Unstructured	https://docs.unstructured.io/	Document partitioning and ingestion support for complex files.
Official Documentation	LangChain	https://python.langchain.com/docs/introduction/	Supports retrieval, structured chains, and tool-connected LLM workflows.
Official Documentation	LangGraph	https://langchain-ai.github.io/langgraph/	Useful for stateful document processing and approval workflows.
Official Documentation	LlamaIndex	https://docs.llamaindex.ai/	RAG framework for indexing and retrieving document knowledge.
Official Documentation	Chroma	https://docs.trychroma.com/	Vector database option for clause libraries and document embeddings.
Official Documentation	Qdrant	https://qdrant.tech/documentation/	Vector database option for retrieval and metadata filtering.
Official Documentation	Supabase	https://supabase.com/docs	Operational database, authentication, storage, and pgvector support.
Official Documentation	FastAPI	https://fastapi.tiangolo.com/	Backend API framework for document workflows and AI service orchestration.
Official Documentation	Next.js	https://nextjs.org/docs	Frontend and API framework for product deployment.
Official Documentation	Vercel	https://vercel.com/docs	Frontend deployment and serverless hosting option.
Official Documentation	Netlify	https://docs.netlify.com/	Frontend deployment option for accessible review environments.
Official Documentation	Model Context Protocol	https://modelcontextprotocol.io/docs	Tool integration pattern for governed AI systems.
GitHub Repository	CrewAI	https://github.com/crewAIInc/crewAI	Alternative multi-agent framework for role-based collaboration workflows.
Official Documentation	Gemini API	https://ai.google.dev/gemini-api/docs	LLM option for structured extraction, reasoning, and summarization.

9. Evaluation Metrics

Category	Marks	What Reviewers Will Look For
Product Understanding and Business Framing	10	Clear understanding of legal and compliance review pain points, business-unit scale, risk exposure, and enterprise value.
Product Experience	12	Quality of upload workspace, workflow visibility, clause review, risk console, summary workspace, and approval experience.
Backend and API Implementation	12	Document upload APIs, processing endpoints, retrieval services, review actions, audit logging, error handling, and secure integration.
AI Workflow Implementation	16	OCR-to-parsing-to-clause-extraction-to-RAG-to-legal-reasoning-to-summarization workflow implemented with clear stages.
Data, Retrieval, and Knowledge Layer	12	Structured clause records, document metadata, vector database, approved references, risk taxonomy, and retrieval traceability.
Reliability, Safety, and Governance	12	Evidence-grounded outputs, confidence indicators, human approval, audit trail, secure storage, and safe failure behavior.
Deployment Readiness	8	Working deployed application, environment variables, stable demo path, and accessible review URL.
Documentation and Repository Quality	8	README, setup instructions, architecture notes, API details, screenshots, schemas, and security notes.
Demo Quality	5	Clear walkthrough from document upload to clause extraction, risk flags, summary, and human approval.
Team Collaboration	5	Clear ownership across product, legal, compliance, frontend, backend, AI, data, QA, security, and documentation.

10. Final Deliverables

- Complete GitHub repository link containing the full codebase, documentation, setup instructions, schemas, and architecture notes.
- Deployed application link clearly mentioned inside the README file.
- Working demo video uploaded to Google Drive with 'Anyone with the link can view' access enabled.
- README with product overview, setup steps, architecture, screenshots, product explanation, API details, workflow explanation, security notes, and team contribution summary.
- Required sample documents, clause library, policy references, risk taxonomy, prompt templates, output schemas, and deployment notes.

11. Final Submission Checklist

- ☐ GitHub repository link is accessible.
- ☐ Complete codebase is available with clear folder structure.
- ☐ README is complete and includes setup, architecture, screenshots, API details, and security notes.
- ☐ Deployed application link is added inside the README.
- ☐ Deployed application is working and reviewable.
- ☐ Core workflow works end to end: upload documents -> OCR -> parsing -> clause extraction -> RAG retrieval -> legal reasoning -> summarization.
- ☐ AI workflow is implemented with clear document-processing and review stages.
- ☐ Data, retrieval, and knowledge layers are working where applicable.
- ☐ Human approval workflow is implemented and visible.
- ☐ Risk flags include rationale, severity, evidence, confidence, and reviewer action state.
- ☐ References and documentation are included.
- ☐ Demo video is accessible with anyone-view access.
- ☐ Team contribution is clearly mentioned.

12. Closing Note

The AI Legal Document Intelligence System is intended to operate as a serious internal legal technology product for Tata Group. Its value depends on precise document ingestion, reliable OCR, structured clause extraction, grounded retrieval, controlled legal reasoning, transparent risk flags, and human approval.